

机器人竞技赛攻防策略的动力学建模与序贯决策优化研究

摘要

机器人竞技，是具身智能在动态对抗场景中综合能力的重要展示平台。攻防策略优化问题，不仅涉及多体动力学控制与实时动作执行，更需要在有限体力、多变距离和实时分差下完成序贯决策与资源调度，对机器人的感知、决策与控制能力提出了极高要求。[1]在“十五五”规划中，明确表明具身智能在推动未来经济增长中发挥关键作用。而目前人形机器人竞技中，常用的固定连招和预设战术，很难应对比赛中的复杂博弈，往往无法充分发挥机器人的硬件性能和战术潜力。本文以众擎 PM01 人形机器人单场竞技赛为研究对象，从单动作的动力学性能量化入手，通过攻防动作收益评估，最终构建全局序贯策略优化模型，系统分析体力、距离和分差等关键状态对战术决策的影响，求解并验证不同竞技阶段下的最优攻防动作序列，为机器人实现自适应自主作战决策提供数学支撑与可直接落地的战术方案。

针对问题一，为完成问题一“量化攻击效果→量化稳定性→综合评价排序”三层递进分析，我们采用拉格朗日动力学模型提取有效动能（主指标）、ZMP 稳定裕度量化稳定性，结合熵权-TOPSIS 综合评价法，完成 13 种攻击动作的综合排序，确定左直拳、右直拳为综合性能最优的攻击动作。

针对问题二，将机器人攻防对抗抽象为双人有限零和博弈，构建 13×22 攻防综合效用矩阵，基于极大极小值准则与纳什均衡理论求解，为每种攻击动作匹配最优防御动作，明确拳法适配格挡类防御、腿法适配闪避类防御的规律。

针对问题三，建立序贯决策特征识别→MDP 模型建立→专家规则构建转移矩阵→值迭代求解→蒙特卡洛验证”的分析链条。将单场竞技赛抽象为马尔可夫决策过程，构建攻防一体化动作空间，采用专家规则估计状态转移概率矩阵，通过值迭代算法求解 Bellman 最优方程获得全局最优策略，并利用蒙特卡洛模拟验证策略效用，最终呈现出“优势求稳、劣势求变”的动态调整规律。

针对问题四，在问题三基准策略基础上引入故障风险约束，构建有限资源的 B03 赛制动态规划模型，以问题三 MDP 输出为输入，通过 Bellman 方程定义状态价值函数，采用近似值迭代与蒙特卡洛采样求解，输出最优资源使用边界，最大化 B03 系列赛获胜概率。最优策略呈现出清晰特征：优势时优先省资源，劣势时集中投入；轻度故障可继续作战，中重度故障在关键局果断使用资源，相比无资源基线策略，胜率提升显著。

针对问题五，综合前述建模结论，完成“模型启示提炼→应用场景映射→建议框架构建”的分析。据此撰写建议书，提出促进我国具身智能领域发展的合理建议。

关键词：拉格朗日动力学、ZMP 稳定裕度、零和博弈、马尔可夫决策过程、随机动态规划、攻防策略优化

一. 问题重述

现依据众擎 PM01 人形机器人在竞技赛中不同动作的攻击与防守效果，尝试建立数据模型解决以下问题：

1. 通过仿真量化机器人每种动作的冲击效果与对自身稳定性的影响，并结合攻击力度与自身平衡之间存在相互制约关系的特征，对附录一所述的 13 种攻击动作（含拳法、腿法、组合技三类）建立动力学模型，从而选取出效果优先的攻击动作，并排出其攻击效果的强弱次序。
2. 为形成防守策略与攻击动作的有效闭环，针对附录二的 22 种防守动作（以格挡、闪避、防御姿态、平衡恢复、倒地反击五类为主），建立最佳防守模型。即针对问题一中每种特定攻击，须从防守动作库中匹配出最优的应对组合，而非孤立评价单一防守的优劣。
3. 在不考虑故障的前提下，将攻击与防守动作按时间序列协同编排，建立单场竞技赛的策略优化模型，使得模型能够输出完整的最佳作战决策方案，实现取胜概率最大化。
4. 考虑到机器人运行中存在故障风险，和动态权衡资源消耗与比赛节奏之间的博弈关系。在赛事规则约束下，建立资源使用的最优时机与次数决策模型，以实现不同赛况下的 B03 三局两胜制最终胜率最大化。
5. 基于上述模型与分析结论，结合未来应用场景，撰写建议书，为我国具身智能领域发展提出合理建议。

二. 问题分析

2.1 问题一的分析

本题的核心问题在于平衡攻击力度与自身平衡之间存在的相互制约关系，并可将该问题分解为三个子问题：如何量化攻击效果，如果量化稳定性，以及如果判定二者间的权重？基于以上认识，我们认为拉格朗日动力学模型是研究机器人动力学模型的常用模型[2]，可量化攻击效果；ZMP 和 COG 是机器人行走稳定性的判定依据，但因“步速提高时机器人处于步行动态，惯性力增强”的事实，排除 COG 方法，选用 ZMP 来判断机器人系统的稳定性[3]，最后采取熵权法对 13 种动作在两指标上的差异自动分配权重，再以 TOPSIS 法计算各动作与理想解的相对贴近度，实现客观排序。

2.2 问题二的分析

本体核心旨在完成机器人攻防动作的最优匹配，实现“攻击动作→最优防守动作”的一一对应。本质来看，机器人攻防对抗可抽象为“双人有限零和博弈”模型，将 13 种攻击、22 种防守视为有限集合，构建 13×22 的攻防综合效用矩阵，量化攻防匹配的效果。运用极大极小值准则求解“防守方最大化最小收益”（即最小化防守损失），再通过纳什均衡验证，确定每种攻击动作对应的最优防守动作。

2.3 问题三的分析

该问题的本质是不确定环境下的序贯决策优化问题，考虑到该问题具有“序贯决策性，状态转移不确定性，长期收益最大化的”三大特征，与马尔可夫决策过程

(MDP) 高度契合，因此我们选用 MDP 模型作为模型框架。由于无法找到物理引擎，我们转向基于专家规则的转移概率矩阵构造方法计算显式的转移概率和奖励，接着通过值迭代求解最优策略，最后设计蒙特卡洛评估的实验方案验证结果。

2.4 问题四的分析

问题四在问题三无故障基准策略的基础上，引入故障风险与三类有限资源约束，要求优化 BO3 三局两胜赛制下的资源使用决策以最大化最终获胜概率，其暗含的随机性, 序贯性, 多阶段耦合性, 符合随机动态规划的条件。因此我们选用随机动态规划模型建立模型，由问题三 MDP 输出作为统计输入，再通过值函数的递归 Bellman 方程定义。后采取近似值迭代，蒙特卡洛采样求解模型。

2.5 问题五的分析

撰写具身智能机器人产业发展建议书，从技术研发、赛事应用与产业推广三个维度提出政策建议。

三. 模型假设

- 假设 1：机器人质量分布与几何参数以众擎 PM01 公开数据为准，单场竞技中不发生永久性结构变形。
- 假设 2：比赛场地平整刚性，摩擦系数恒定，排除局部凹陷等情况。
- 假设 4：机器人各连杆视为刚体，忽略间隙与摩擦阻尼。
- 假设 5：各攻击动作的关节驱动力矩按预设曲线执行，动作间相互独立，不考虑电机饱和与延迟。
- 假设 6：单次攻击动作期间仅与地面接触，攻击末端触靶前与目标无预接触。
- 假设 7：攻击动作动力学分析中不考虑对手实时反击的力学干扰，对手交互影响在后续问题中处理。
- 假设 8：机器人竞技赛的状态转移具有无后效性，各状态可完全观察，状态转移概率基于专家规则构建的先验值合理有效，忽略对手策略突变对状态转移的极端影响。

四. 定义与符号说明

符号	说明	符号	说明
q_j	第 j 个关节的转角或末端执行器的空间位移	S	状态空间
$\dot{q}^k$	广义速度	A	己方/对手血量等级
$\ddot{q}^k$	广义加速度	h_s, h_o	己方/对手姿态
V^k	势能	d	相对距离
$M(q)$	关节空间质量矩阵，反映各连杆的惯性特性及耦合效应	ϕ	回合时间步

$C(q, \dot{q})\dot{q}$	科里奥利力与离心力项	$a = (a_{atk}, a_{def})$	攻击-防守动作对
$G(q)$	重力项	$P(s' s, a)$	状态转移概率
$\tau^k(t)$	第 k 种动作的关节驱动力矩（已知的运动控制输入）	$R(s, a, s')$	即时奖励函数
$J_{end}(q)$	攻击末端（拳部或足部质心）的雅可比矩阵	γ	折扣因子
$M(q)$	拉格朗日方程的质量矩阵	$V^*(s)$	最优状态值函数
$F_{GRF,z}$	地面法向反力	$\pi^*(s)$	最优策略
$M_{GRF,x}$	绕 x 轴的地面反力矩	w_m	我方已胜局数
$M_{GRF,y}$	绕 y 轴的地面反力矩	w_o	对方已胜局数
(x_l, y_l)	第 l 个接触点的地面投影坐标	r_{res}	人工复位剩余次数
H_{ij}	攻击 k_i 对阵防御 d_j 造成有效伤害值	r_{tact}	战术暂停剩余次数
C_i	执行攻击动作 k_i 产生能耗	r_{repair}	紧急维修剩余次数
R_{ij}	该攻防组合下我方被反击成本 风险值	h	紧急维修剩余次数
s_{diff}	本局比分差	f	当前故障等级

表 1

五. 模型的建立与求解

5.1 问题一的模型

5.1.1 模型建立

一. 基于拉格朗日动力学的攻击动作力学模型

1. 定义机器人的广义坐标

将众擎 PM01 机器人抽象为具有 n 个自由度的多刚体系统，对于 13 种攻击动作中的第 k 种动作（k=1, 2...13），选定其广义坐标为：

$$q^k = (q_1, q_2 \dots q_n)^T$$

2. 运用拉格朗日运动方程

系统的拉格朗日量为动能与势能之差表示为： $L^k = T^k(q, \dot{q}) - V^k(q)$

根据欧拉-拉格朗日方程导出动力学方程的显式形式：

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau^k(t)$$

通过数值积分，得出该方程每一时刻所有关节的 $q(t)$, $\dot{q}(t)$, $\ddot{q}(t)$ 及 τ^t ,

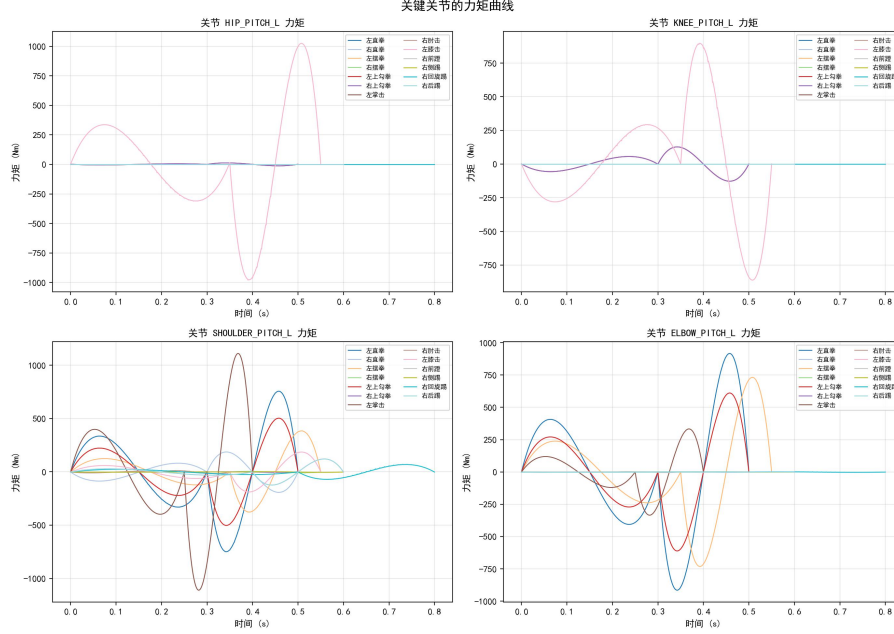


图 1 攻击动作在完整运动周期内各关键关节的力矩曲线

3.从拉格朗日方程直接提取攻击效果指标

下面从拉格朗日方程中直接构建三个递进的物理量：

(1) 末端线速度

基于关节速度与雅可比矩阵，将关节速度映射为笛卡尔空间线速度：

$$v_{end}^k(t) = J_{end}(q) \cdot \dot{q}^k(t)$$

(2) 等效质量

单纯的速度忽略了身体其余环节的惯性耦合，定义笛卡尔空间惯性矩阵：

$$A_{v(q)} = [J_{end}(q) \cdot M(q)^{-1} \cdot J_{end}(q)^T]^{-1}$$

在攻击方向单位向量 u （由打击目标方向定义）上的等效质量为：

$$m_e^k = \frac{1}{u^T [\Lambda_v(q(t_{hit}^k))]^{-1} u}$$

(3) 有效动能与有效动量（主指标）

$$E_k^k = \frac{1}{2} m_e^k [v_{end}^k]^2 p^k = m_e^k \cdot v_{end}^k$$

4.攻击效果得分归一化

$$S_{attack}^k = \frac{E_k^k - E_k^{\min}}{E_k^{\max} - E_k^{\min}}$$

其中 $E_k^{\max} = \max_k E_k^k, E_k^{\min} = \min_k E_k^k$ 。 $S_{attack}^k \in [0,1]$ ，值越大表示攻击效果越强。

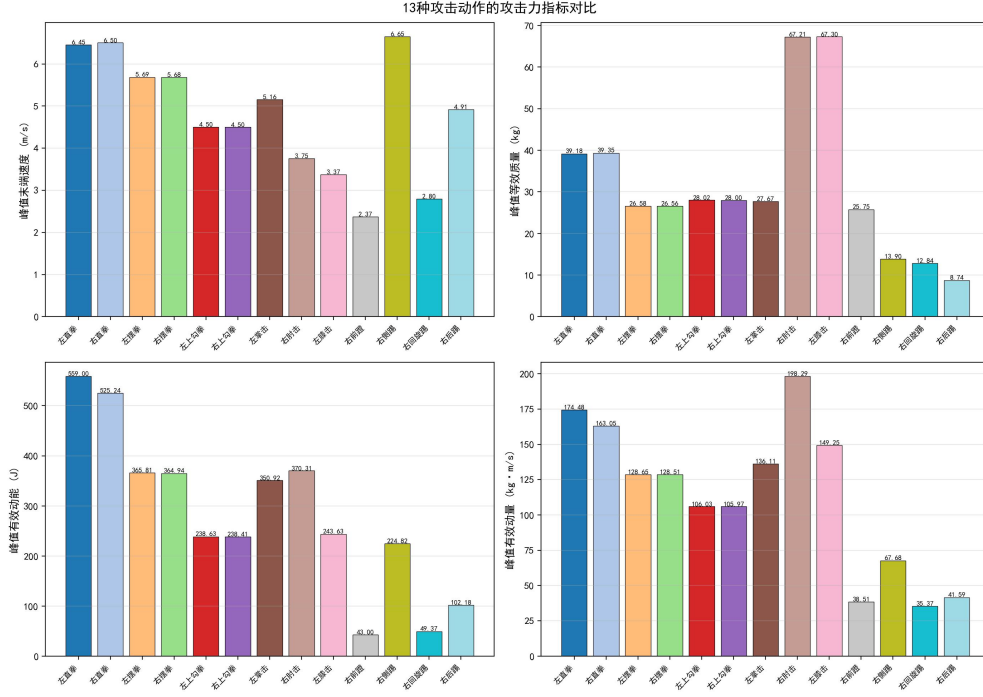


图 2 13 种攻击动作效果指标对比

二. 基于 ZMP 理论的稳定性约束模型

1. 足底反力计算

ZMP 的计算需要足底反力这一数据, 该数据可基于拉格朗日动力学方程输出的关节力矩, 通过牛顿 - 欧拉递推法反推出足底反力与踝关节力矩。

2. ZMP 计算

利用反推得到的足底反力, 计算 ZMP 坐标:

$$x_{ZMP}^k(t) = \frac{-M_{GRF,y}}{F_{GBF,z}}, y_{ZMP}^k(t) = \frac{M_{GRF,x}}{F_{GBF,z}}$$

3. 支撑域定义

机器人在执行攻击动作时, 足底与地面接触区域构成支撑多边形:

$$\Omega_{support}^k(t) = Conv\{(x_l, y_l) | l = 1, 2, \dots, N_{contact}\}$$

结合上文算出的 ZMP 坐标, 绘制出各动作的 ZMP 轨迹与支撑多边形, 如图 3 结果表明, 绝大部分拳法动作的 ZMP 轨迹落在支撑多边形内, 而部分腿法动作 (如右侧踢) 的 ZMP 轨迹超出了支撑范围, 存在较大风险。

4. ZMP 稳定裕度 (稳定性主指标)

ZMP 到支撑域边界的最小距离:

$$d_{ZMP}^k(t) = \min_{(x,y) \in \partial\Omega_{support}^k(t)} \|(x_{ZMP}^k(t), y_{ZMP}^k(t)) - (x, y)\|$$

$d_{ZMP} > 0$ 时，ZMP 在支撑域内，机器人动态稳定； $d_{ZMP} \leq 0$ 时，ZMP 超出支撑域，存在倾倒风险。

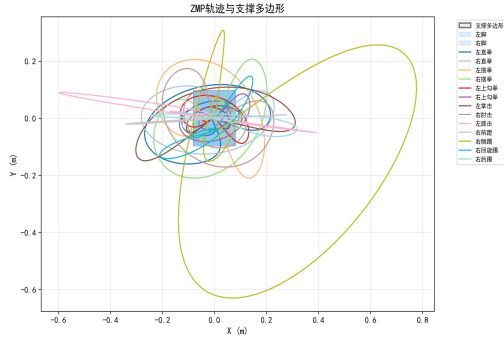


图 3 ZMP 轨迹与支撑多边形

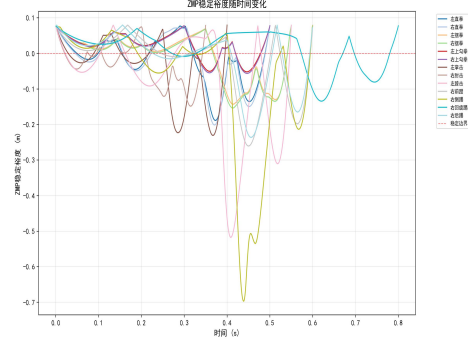


图 4 ZMP 稳定裕度随时间变化

图中显示拳法动作的稳定裕度整体波动较小，大部分时间维持在 0 值附近，整体稳定性表现良好；而腿法动作的稳定裕度在发力阶段出现极具下跌，尤其是右侧踢、左膝击的裕度最低值接近 -0.7m，极大概率倾倒。

5. 稳定性指标提取策略

根据优化准则，应取动作执行全程的最危险时刻进行评估：

$$\tilde{d}^k = \min_{t \in [t_0^k, t_f^k]} d_{ZMP}^k(t)$$

结合有效动能（攻击效果主指标）与 ZMP 最小稳定裕度（稳定性主指标），可构建攻击动作的帕累托前沿。

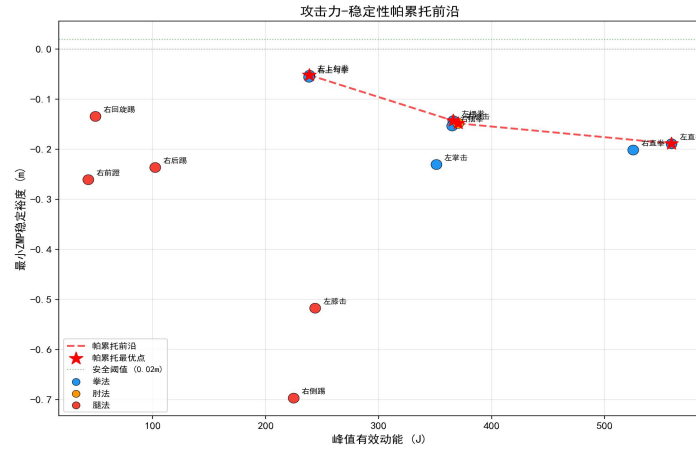


图 5

图中红色虚线构成的帕累托前沿清晰揭示了攻击动作的性能与稳定性权衡关系，拳法与肘法动作（蓝橘色点）整体处于前沿之上，是兼顾攻击效果与稳定性的最优选择，其中左右直拳兼具高动能与高稳定性，是最优选择；而腿法动作存在存在性能与稳定的失衡。

6. 稳定性得分归一化

$$S_{stab}^k = \frac{\tilde{d}^k - \tilde{d}^{\min}}{\tilde{d}^{\max} - \tilde{d}^{\min}}$$

其中 $\tilde{d}^{\max} = \max_k \tilde{d}^k, \tilde{d}^{\min} = \min_k \tilde{d}^k$ 。 $S_{\text{stab}}^k \in [0,1]$ 值越大表示稳定性越好。

三. 基于 TOPSIS 熵权法的双目标综合评价

1. 决策矩阵构建

对于 13 种攻击动作，每个动作有两个正向评价指标：

$$X = [x_{ij}]_{13 \times 2}, x_{i1} = S_{\text{attack}}^i, x_{i2} = S_{\text{stab}}^i, i = 1, \dots, 13$$

2. 数据标准化

采用极差变换法进行正向标准化：

$$r_{ij} = \frac{x_{ij} - \min_i x_{ij}}{\max_i x_{ij} - \min_i x_{ij}}$$

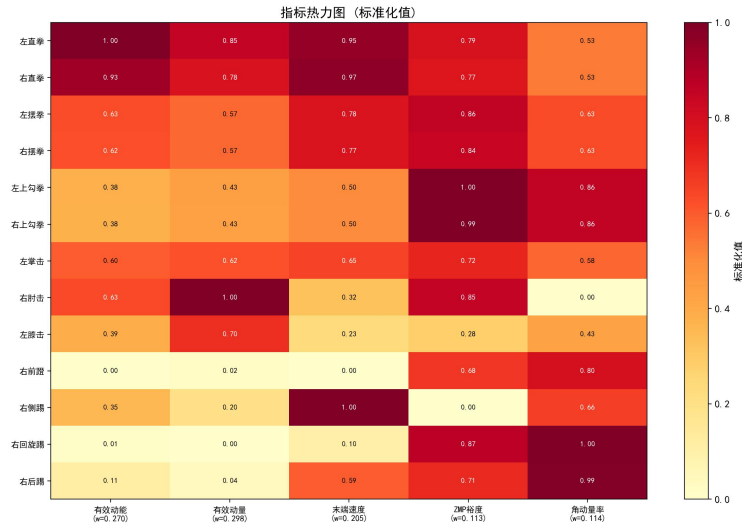


图 6 所有数据取标准化结果

3. 熵权法客观赋权

计算第 j 个指标下第 i 个动作的特征比重： $p_{ij} = \frac{r_{ij}}{\sum_{i=1}^{13} r_{ij}}, j = 1, 2$

计算第 j 个指标的信息熵： $E_j = -\frac{1}{\ln 13} \sum_{i=1}^{13} p_{ij} \ln(p_{ij})$

计算第 j 个指标的熵权： $w_j = \frac{1 - E_j}{2 - (E_1 + E_2)}$

熵权法依据各动作在攻击效果和稳定性上表现出的内在差异自动分配权重。具体熵权分布显示：有效动量（29.75%）、有效动能（27.00%）、末端速度（20.55%）是影响攻击性能的三大主导因素，角动量率（11.37%）、ZMP 裕度（11.33%）可看作辅助贡献指标。

4. TOPSIS 综合评价

加权标准化： $v_{ij} = w_j \cdot r_{ij}$

正理想解： $A^+ = \{\max_i v_{i1}, \max_i v_{i2}\}$ 负理想解： $A^- = \{\min_i v_{i1}, \min_i v_{i2}\}$

欧氏距离: $D_i^+ = \sqrt{\sum_j (v_{ij} - v_j^+)^2}, D_i^- = \sqrt{\sum_j (v_{ij} - v_j^-)^2}$

相对贴近度: $C_i = \frac{D_i^-}{D_i^+ + D_i^-}, i = 1, 2, \dots, 13$ C_i 越大, 该动作在攻击与稳定的综合越好。

5.动作优选与排序

将 13 种动作按 C_i 降序排列, 并取 $C_i \geq \bar{C}$ ($\bar{C}$ 为 13 个动作的均值) 的动作作为推荐攻击组合, 攻击效果强弱的排序即为 C_i 的降序排列。

5.1.2 模型解答与分析

最终排列结果如下表所示:

动作名称	峰值速度 (m/s)	等效质量 (kg)	有效动能 (J)	有效动量 (kg·m/s)	ZMP裕度 (m)	TOPSIS 评分	排名
左直拳	6.45	39.2	559.0	174.5	-0.1885	0.8554	#1
右直拳	6.50	39.3	525.2	163.0	-0.2012	0.8212	#2
左摆拳	5.69	26.6	365.8	128.7	-0.1429	0.6413	#3
右摆拳	5.68	26.6	364.9	128.5	-0.1535	0.6395	#4
左上勾拳	4.50	28.0	238.6	106.0	-0.0517	0.4853	#7
右上勾拳	4.50	28.0	238.4	106.0	-0.0561	0.4846	#8
左掌击	5.16	27.7	350.9	136.1	-0.2306	0.6205	#6
右肘击	3.75	67.2	370.3	198.3	-0.1483	0.6381	#5
左膝击	3.37	67.3	243.6	149.3	-0.5173	0.4789	#9
右前踢	2.37	25.8	43.0	38.5	-0.2604	0.2090	#13
右侧踢	6.65	13.9	224.8	67.7	-0.6968	0.4352	#10
右回旋踢	2.80	12.8	49.4	35.4	-0.1342	0.2566	#12
右后踢	4.91	8.7	102.2	41.6	-0.2361	0.3286	#11

图 7

因此基于 TOPSIS 多指标综合评价模型, 13 种攻击动作的性能排序结果显示, 优先选择左直拳、右直拳、左摆拳、右摆拳这四类动作。其中左直拳以 0.8554 的 TOPSIS 评分位居第一, 其有效动能高达 559.0J, 同时保持了良好的动态稳定性, 是兼顾攻击效果与机器人稳定性的最优选择; 右直拳、左摆拳与右摆拳则分别在攻击输出、稳定性与战术多样性上形成互补, 共同构成机器人攻击动作的核心动作库。肘击与腿法动作虽具备一定效果, 但各有短板, 综合表现不及拳法动作。

5.2 问题二的模型

5.2.1 模型建立

1.构建攻击 - 防御效用矩阵

设我方共 13 种攻击动作: $K = \{k_1, k_2 \dots k_{13}\}$

敌方共 22 种防御动作: $D = \{d_1, d_2 \dots k_{22}\}$

基于三维指标, 运用单组攻防组合综合收益公式: $U_{ij} = H_{ij} - C_i - R_{ij}$
由此构建 13 行 22 列攻击 - 防御效用矩阵:

$$U = \begin{bmatrix} U_{11} & U_{12} & \cdots & U_{1,22} \\ U_{21} & U_{22} & \cdots & U_{2,22} \\ \vdots & \vdots & \ddots & \vdots \\ U_{13,1} & U_{13,2} & \cdots & U_{13,22} \end{bmatrix}$$

矩阵行代表攻击策略，列代表防御策略，矩阵元素为该策略组合下整体收益。

2.运用双人零和博弈模型

机器人攻防对抗分为进攻方（我方）、防守方（敌方），其中我方 13 种攻击动作、敌方有 22 种防御动作，均为有限离散策略。我方追求综合收益最大化，敌方追求自身损失最小化，且我方的收益即为敌方的损失，满足零和博弈的“总收益和为 0”。 [4]

基于以上特征符合双人有限零和博弈的条件。所以为匹配 “为每种攻击动作确定最佳防御动作” 的目标，我们采用零和博弈模型求解。

5.2.2 模型求解

零和博弈下，防守方的最优策略是“最坏情况下损失最小”，即极大极小准则，适配“单种攻击对应固定最佳防御”的需求。

(1) 进攻方视角

对于我方的任意攻击动作 k_i ，防守方会选择使我方收益最小的防御动作，即：

$$\min_{d_j \in D} U(k_i, d_j)$$

(2) 防守方视角

针对我方的攻击动作 k_i ，敌方的最优防御动作 d_j^* 满足： $d_j^* = \arg \min_{d_j \in D} U(k_i, d_j)$

即：在所有防御动作中，选择使我方综合收益 $U(k_i, d_j)$ 最小的动作，此时敌方的损失最小，为该攻击下的最佳防御策略。

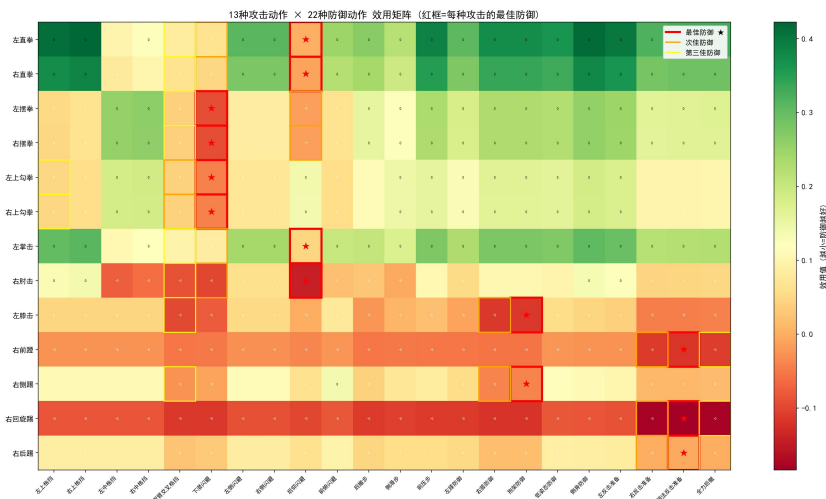


图 8

5.2.3 模型解答与分析

攻击动作	最佳防御1	U值	最佳防御2	U值	最佳防御3	U值
左直拳	后仰闪避	0.0	下潜闪避	0.1	双臂交叉格挡	0.1
右直拳	后仰闪避	-0.0	下潜闪避	0.0	双臂交叉格挡	0.1
左摆拳	下潜闪避	-0.1	后仰闪避	-0.0	双臂交叉格挡	0.0
右摆拳	下潜闪避	-0.1	后仰闪避	-0.0	双臂交叉格挡	0.0
左上勾拳	下潜闪避	-0.0	双臂交叉格挡	0.0	左上格挡	0.0
右上勾拳	下潜闪避	-0.0	双臂交叉格挡	0.0	左上格挡	0.0
左掌击	后仰闪避	0.0	下潜闪避	0.1	双臂交叉格挡	0.1
右肘击	后仰闪避	-0.1	下潜闪避	-0.1	双臂交叉格挡	-0.1
左膝击	抱架防御	-0.1	右膝防御	-0.1	双臂交叉格挡	-0.1
右前踢	腿法反击准备	-0.1	右反击准备	-0.1	全力后撤	-0.1
右侧踢	抱架防御	-0.0	右膝防御	-0.0	双臂交叉格挡	-0.0
右回旋踢	腿法反击准备	-0.2	右反击准备	-0.2	全力后撤	-0.2
右后踢	腿法反击准备	-0.0	右反击准备	-0.0	全力后撤	0.0

图 9

观察结果，可总结出“拳法优先闪避、腿法优先后撤、肘膝类攻击需提前准备反击”的策略原则。

5.3 问题三的模型

5.3.1 模型建立

马尔可夫决策过程适用的系统有三大特点：一是状态转移的无后效性，二是状态转移可以有不确定性，三是智能体所处的每步状态完全可以观察。[5] 以上特点符合该题特征，因此我们建立 MDP 基本数学模型。

1. 状态空间 S

将单场竞技赛的状态定义为六元组： $S = (h_s, h_o, p_s, p_o, d, \phi)$

各分量定义如下：

分量	解释
血量等级 h_s, h_o	将原始血量归一化后均匀离散化为 H+1 级（0 表示被 KO，H 表示满血）。取 H=5，则每级跨度约为原始血量的 20%
姿态 p_s, p_o	编码为 0（站立）、1（失衡/硬直）、2（倒地）。姿态决定可用动作集合与受击判定
相对距离 d	编码为 0（近距，可直接拳击）、1（中距，腿法可达）、2（远距，需位移逼近）
回合时间 ϕ	从比赛开始累计的时间步， $\phi = \Phi$ 时回合结束进入判定。取 $\Phi = 100$

表 2

状态空间总规模： $|S| = 6^2 \times 3^2 \times 3 \times 101 \approx 9.8 \times 10^4$

2. 动作空间 A

基于问题一给出 N_a 种优选攻击动作及其攻击效果得分 S_{attack}^i ，问题二为每种攻击匹配了 N_d 种防守动作。构建攻防一体化动作空间：

$$A = \{a_{\text{atk}}, a_{\text{def}} \mid a_{\text{atk}} \in A_{\text{atk}}, a_{\text{def}} \in A_{\text{def}}\}, \text{ 取 } N_a = N_d = 6, \text{ 则 } |A| = 36$$

3. 状态转移概率矩阵 $P(s' | s, a)$

转移概率 $P(s' | s, a)$ 是 MDP 的核心, 描述在状态 S 执行动作 a 后转移到 s' 的概率。由于对手策略未知, 基于专家规则构建先验转移概率。假设各维度的转移条件独立, 将联合转移概率分解为: $P(s' | s, a) = P_h P_d P_p$ [6]

5.3.2 模型求解

基于上述模型得出显式的转移概率和奖励, 即可用值迭代求解最优状态价值函数 $V^*(s)$ 和最优策略 $\pi^*(s)$ 。

首先初始化所有状态价值 $V(s) = 0$

在 $|S| \approx 9.8 \times 10^4, |A| = 36$ 的设置下, 值重复迭代直到 $\max |V_{k+1} - V_k| < 10^{-4}$, 根据收敛后的 $V^*(s)$ 输出最优策略 $\pi^*(s)$

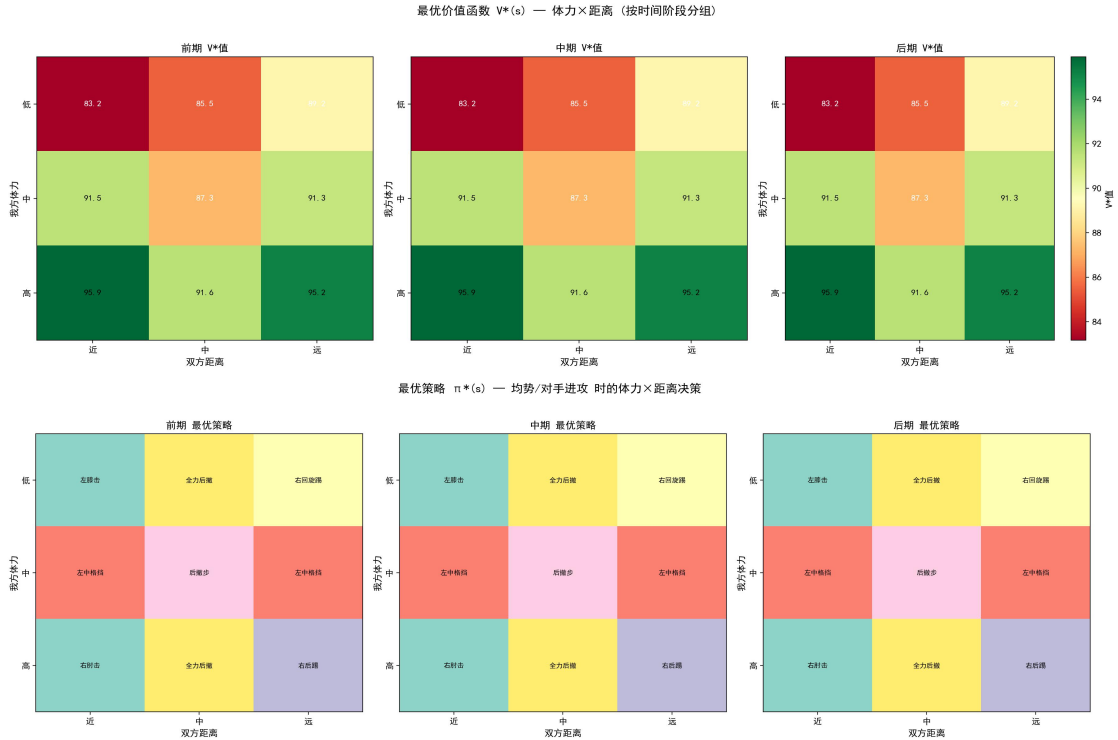


图 10

值迭代收敛后，考虑在不同状态下（不同距离，不同分差，不同体力水平）的最优动作的映射表

通过分析结果，可总结出机器人最佳攻防序列：

回合阶段	条件（体力，分差，距离）	最佳攻击动作	最佳防御动作
第一回合开局	高，均势，中 / 远	左路击	左中格挡
第一回合中段	中，均势，近	左中格挡	左中格挡
第一回合末	中，优势，中	左排击	后撤步
第二回合	中，劣势，近	左中格挡	右回旋踢
第三回合	低，任何，近	左腿击	左腿击

表 3

并给出机器人最佳的作战决策方案：

分差	体力	距离	最优动作	类型	V*值
劣势	低	近	左膝击	攻击	90.9
劣势	低	中	全力后撤	防御	94.0
劣势	低	远	右回旋踢	攻击	96.6
劣势	中	近	左中格挡	防御	101.1
劣势	中	中	左中格挡	防御	95.8
劣势	中	远	左中格挡	防御	99.3
劣势	高	近	右肘击	攻击	104.4
劣势	高	中	左摆拳	攻击	100.6
劣势	高	远	右后踢	攻击	102.8
均势	低	近	左膝击	攻击	84.9
均势	低	中	全力后撤	防御	87.1
均势	低	远	右回旋踢	攻击	90.5
均势	中	近	左中格挡	防御	93.2
均势	中	中	后撤步	防御	88.1
均势	中	远	左中格挡	防御	92.1
均势	高	近	右肘击	攻击	96.6
均势	高	中	全力后撤	防御	92.9
均势	高	远	右后踢	攻击	96.7
优势	低	近	全力后撤	防御	76.9
优势	低	中	全力后撤	防御	80.6
优势	低	远	左反击准备	防御	81.9
优势	中	近	左中格挡	防御	85.5
优势	中	中	左中格挡	防御	81.6
优势	中	远	左中格挡	防御	85.7
优势	高	近	左反击准备	防御	87.6
优势	高	中	全力后撤	防御	86.3
优势	高	远	左反击准备	防御	87.7

图 11

基于以上结果，总结出“优势状态下以稳为主，优先防御控场；劣势状态下主动进攻，高威胁动作优先；高体力、中远距离下适合采用中距离攻击动作；低体力时更倾向于近距离攻击或防御动作，避免过度消耗”的策略方案。

5.4 问题四的模型

5.4.1 模型建立

将 B03 系列赛中任一决策时刻的状态定义为：

$$S = (w_m, w_o, r_{res}, r_{tact}, r_{repair}, h, f, t, s_{diff})$$

将上述状态空间规模离散化后得总状态数 $S \approx 9 \times 18 \times 5 \times 4 \times 10 \times 5 = 162000$ ，根据以上数据建立模型：

一．故障发生模型（泊松过程）

故障发生时刻服从非齐次泊松过程，强度函数 $\lambda(t, h)$ 与健康度负相关：

$$\lambda(t, h) = \lambda_0 \cdot e^{-ah} (1 + \beta \cdot \text{累计攻击次数})$$

在离散时间步 Δt 内，故障发生概率： $P(\text{fault} | t, h) = 1 - e^{-\lambda(t, h)\Delta t}$ [7]

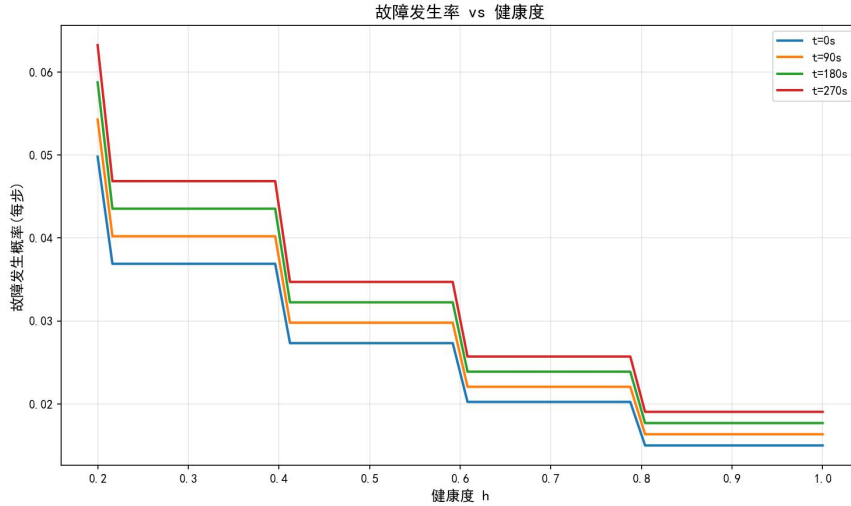


图 12

图中可看出健康度越低，故障概率越高，且随时间不断累积。

二．故障等级演化

若发生故障，等级 f 按以下概率转移：

$$P(f' = j | f = j) = \begin{cases} p_{i,j} & \text{轻微故障} \\ p_{i,i+1} & \text{故障加重} \\ p_{i,0} & \text{维修后恢复} \end{cases}$$

算出资源使用后的状态转移

资源	对状态的影响
人工复位	$f \leftarrow \max(0, f-1)$ ，消耗 30 秒
战术暂停	健康度 $h \leftarrow h + \Delta h$ （小幅度恢复），打断对方节奏
紧急维修	$f \leftarrow 0, h \leftarrow 0.9$ ，耗时 5 分钟

表 4

三. Bellman 最优方程

定义 $V(s)$ 为在状态 s 下的最大获胜概率（本局+后续局）。

设当前状态决策 $S = (w_m, w_o, r_{res}, r_{tact}, r_{repair}, h, f, t, d)$, 决策 $a \in A$:

$A = \{\text{继续, 人工复位, 战术暂停, 紧急维修}\}$

运用 Bellman 方程: $V(s) = \max_{a \in A} \left\{ \sum_{s'} P(s' | s, a) \cdot V(s') \right\}$

设 $V_{\text{end}} = (w_m, w_o, h, f, r_{res}, r_{tact}, r_{repair})$ 为本局结束时的值函数

根据本局胜负判断: 若本局获胜: $w'_m = w_m + 1$

若本局落败: $w'_o = w_o + 1$

判定系列赛终止的条件为 $V_{\text{series}}(w_m, w_o) = \begin{cases} 1, & w_m = 2 \\ 0, & w_o = 2 \\ V_{\text{next_game}}(\dots) & \text{否则} \end{cases}$

接着重置局间状态, 设下一局开始时的状态:

$S_{\text{next}} = (w'_m, w'_o, r_{res}, r_{tact}, r_{repair}, h_{\text{next}}, f_{\text{next}}, t = 5 \text{ min}, d = 0)$

下图展示不同状态下的价值函数, 说明了故障等级、健康度对获胜概率的影响:

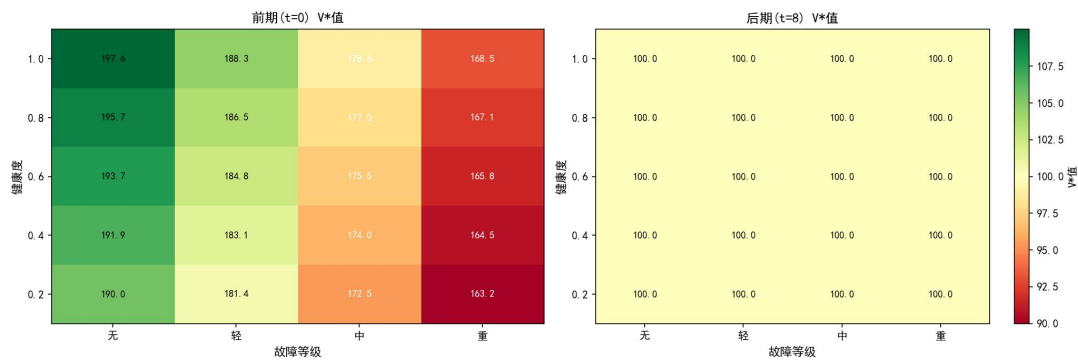


图 13

5.4.2 模型的求解

首先使用迭代框架：

由于连续状态（健康度、时间），采用离散化+插值的近似值迭代

再使用蒙特卡洛采样求解期望：

对每个 (s, a) 采样故障发生时间，攻防对抗结果（基于问题 3 的策略）

，递推计算后续状态值，重复 N 次后取平均。

5.4.3 模型的简答与分析

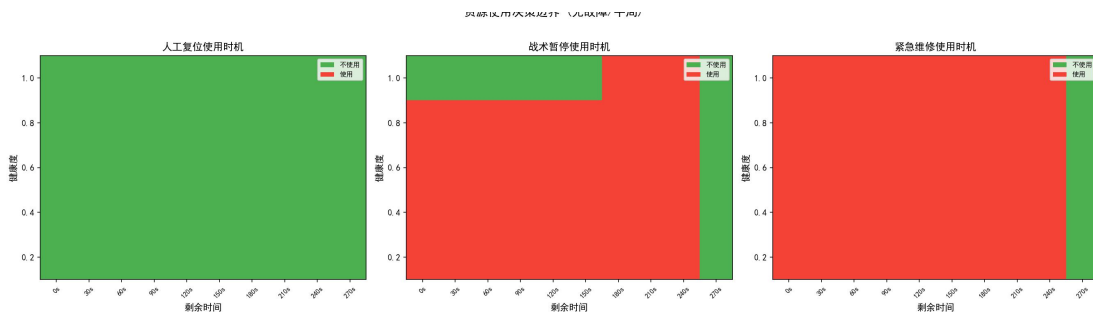


图 14 资源使用决策边界对比图

通过对模型求解出的对比图，总结出以下结论：

资源	最优使用策略（无故障场景）
人工复位	全程不使用
战术暂停	绝大多数场景都使用，仅终局高健康状态不使用
紧急维修	几乎全程使用，仅最后几秒不使用

表 5

并给出机器人最佳的作战决策方案：

803比分	健康度	最优动作	胜率(满资源)	胜率(无资源)	提升
0-0	1.0	人工复位	0.772	0.708	+0.064
0-0	0.6	战术暂停	0.717	0.633	+0.084
0-0	0.2	人工复位	0.660	0.605	+0.055
0-1	1.0	人工复位	0.509	0.422	+0.088
0-1	0.6	人工复位	0.444	0.358	+0.086
0-1	0.2	人工复位	0.404	0.325	+0.078
1-0	1.0	人工复位	0.901	0.878	+0.023
1-0	0.6	战术暂停	0.880	0.840	+0.040
1-0	0.2	人工复位	0.845	0.816	+0.029
1-1	1.0	人工复位	0.722	0.672	+0.050
1-1	0.6	人工复位	0.677	0.627	+0.050
1-1	0.2	人工复位	0.621	0.571	+0.050

图 15

5.5.问题五的解答

数字经济时代，人形机器人产业竞争的焦点正从技术转向生态，而生态的驱动力即为应用场景。只有当技术供给、场景吸收、数据反馈与再创新形成真实场景闭环，并构建起“驱动力—约束力—指引力”的三元生态体系，人形机器人产业才能具备真正的生命力。[8]

本篇论文通过“动力学-博弈-序贯决策-风险管理”的全链条建模，揭示了具身智能从单机性能到系统智能的演进规律。模型结论表明：真正的具身智能，是在物理极限（动力学）、对抗博弈（博弈论）、长期规划（MDP）与风险约束（动态规划）四重维度上，持续逼近最优平衡的能力。基于此提出以下建议。

1. 建立“模型驱动”的底层开发规范。要求动态性能要求高的机器人，必须在控制回路中嵌入拉格朗日动力学前馈，从根源提升运动物理合理性，避免“黑箱”训练带来的生硬与风险。
2. 将 ZMP 稳定裕度定为安全冗余的“金标准”。在服务机器人高速移动、人形机器人跑跳等场景，实时监控 ZMP 到支撑域边界的距离，作为独立安全节点的触发条件。裕度归零立即保护，不等失稳再纠正，筑牢人机共存的安全底线。
1. 建立多智能体博弈测试基准。国家牵头建设标准化对抗仿真环境，要求机器人在合作、竞争等场景下，能稳定收敛到纳什均衡策略或显示序贯最优行为，作为产品智能水平的“能力试金石”。
2. 将“动态策略透明度”纳入评测。要求机器人在执行任务时，能解释其当前策略（如“当前优势，采取保守策略”），这既能建立用户信任，也便于发现决策缺陷，推动策略可解释性的产业化落地。
1. 建立“预测性维护与任务价值”联动机制。在工业等长期运行场景，将机器人健康状态与当前任务价值动态匹配。系统自动判断“继续运行直到完成高价值任务”与“立即停机维修”的最优切换点，最大化整体效益。
2. 推广“风险预算”管理工具。为运营方提供可视化的风险-资源调度界面，内置基于 MDP 和动态规划的决策支持引擎，帮助以最少的维修资源消耗，实现最高的集群任务完成率。
1. 智能制造领域，依托问题一的动力学优化成果，主攻重载精密装配“快且稳”的无损对接；依托问题四的调度逻辑，实现集群机器人的作业节奏最优化。

2. 特种应急领域，依托问题三的序贯决策能力，使机器人在未知地形上自主权衡探索与安全；依托问题二的博弈模型，支撑排爆、处置等对抗任务的策略生成。

3. 养老康复领域，依托问题一的动量-稳定联合监控，以物理硬约束保障搀扶、助行的绝对安全，以问题三的自适应逻辑提供个性化的辅助策略。

综上，我国具身智能产业要实现引领，必须在物理规律理解、博弈策略生成、长期规划调度三个层面深度整合。建议有关方面以此为方向，政策上设立专项攻关与标准制定，产业上打造标杆应用，生态上鼓励开源仿真与数据共享，共同推动具身智能真正从实验室走向千行百业，实现大规模、高可靠、高安全的产业化落地。

六. 模型检验与分析

6.1 问题二的模型检验与分析

选用纯策略纳什均衡验证：

若存在局势 (k_i^*, d_j^*) ，满足：

$$U(k_i^*, d_j^*) = \max_{k_i \in K} \min_{d_j \in D} U(k_i, d_j) = \min_{d_j \in D} \max_{k_i \in K} U(k_i, d_j)$$

则该局势为攻防博弈的纯策略纳什均衡，此时双方均无动机单独改变策略，攻防动作形成稳定的最优对抗搭配。

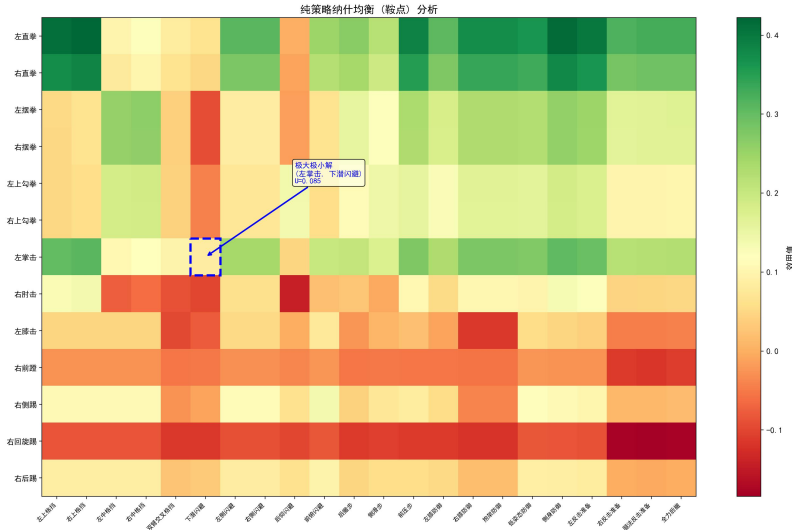


图 16

热力图清晰呈现了所有纯策略组合的效用分布，并且成功定位到了纳什均衡点，不是混合策略的无确定解状态，验证成功。

6.2 问题三的模型检验与分析

用蒙特卡洛仿真评估最终策略：假设对手采用随机均匀策略（从合法动作中等概率选择），进行 10000 次模拟结果并统计胜率，通过最优策略的期望累计奖励验证策略的有效性。

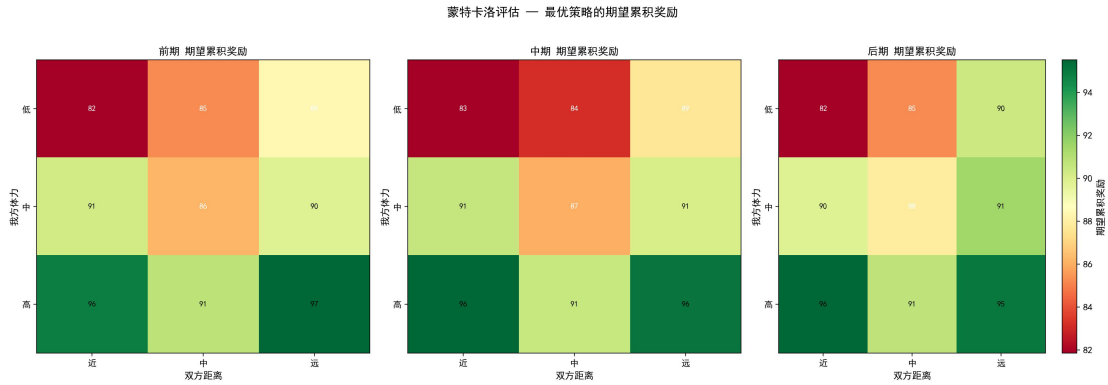


图 17

结果显示平均奖励达 89.72，表明该最优策略是成功的，尤其适合高体力状态下的快节奏攻防，但低体力状态存在一定不稳定性。

七. 模型评价与推广

7.1 模型一的评价与推广

7.1.1 模型一的优缺点

基于拉格朗日动力学和 ZMP 理论，结合熵权 TOPSIS 法，可量化攻击效果与稳定性，筛选结果科学可解释，但未考虑实际环境干扰与关节损耗，且未纳入战术适配性指标，适用场景和评价维度不够全面。

7.1.2 模型一的推广

可推广至各类多自由度机器人的动作规划与性能评价，适配服务、工业及仿生机器人等场景

7.2 模型二的评价与推广

7.2.1 模型二的优缺点

构建攻防效用矩阵量化策略收益，结合双人零和博弈极大极小准则，精准匹配“为每种攻击动作确定最佳防御动作”的目标，求解逻辑贴合问题二攻防对抗场景，但未考虑攻防动作的概率性选择，仅求解固定最优防御，未兼顾敌方防御策略的随机性，适配性有限。

7.2.2 模型二的推广

可推广至各类双人对抗场景的策略匹配问题，适配机器人攻防、竞技对抗等同类策略优化场景。

7.3 模型三的评价与推广

7.3.1 模型三的优点

基于 MDP 模型适配竞技赛状态无后效性、转移不确定性特征，构建六元组状态空间与攻防一体化动作空间，结合值迭代求解最优策略，可输出不同状态下的最佳攻防序列，贴合问题三作战决策需求。

模型缺点：状态转移概率基于专家规则构建先验值，未结合实际竞技数据校准，且未考虑对手动作偏好，可能导致策略适配性不足。

7.3.2 模型三的推广

可推广至各类竞技对抗、智能决策场景，适配机器人格斗、竞技体育战术规划等同类状态决策问题。

7.4 模型四的评价与推广

7.4.1 模型四的优缺点

结合非齐次泊松过程构建故障发生模型，融合故障等级演化与 Bellman 最优方程，采用离散化插值迭代与蒙特卡洛采样求解，可输出 B03 系列赛中资源最优使用策略，贴合问题四故障与资源决策需求；但故障发生强度函数仅与健康度、时间相关，未考虑动作强度对故障的影响，且蒙特卡洛采样的迭代精度受采样次数限制。

7.4.2 模型四的推广

可推广至各类带故障干扰、资源约束的多回合对抗场景，适配机器人系列赛、竞技赛事等同类决策问题。

八. 参考文献

- 【1】人形机器人产业的应用场景，生态短板与未来指向-郭颖
- 【2】SCARA 机器人的拉格朗日动力学建模-崔敏其
- 【3】魏扬帆，周川，郭健，等 . 基于 CPG 的四足机器人坡面稳定行走控制研究 [J] . 控制工程，2021，28（6）：1055-1060.WEI Yangfan, ZHOU Chuan, GUO Jian, et al. CPG-based stablewalking control of the quadruped robot on the slope [J] . ControlEngineering of China, 2021, 28（6）：1055-1060
- 【4】An Overview of Multi-agent Reinforcement Learning from Game Theoretical Perspective Yaodong Yang* 1,2 and Jun Wang1,21University College London, 2Huawei R&D U.K
- 【5】基于马尔科夫决策理论的规范问题的研究-范常杰
- 【6】Humanoid Locomotion and Manipulation: Current Progress and Challenges in Control, Planning, and Learning -Zhaoyuan Gu1, Junheng Li2, Wenlan Shen3, Wenhao Yu4, Zhaoming Xie5, Stephen McCrory6, Xianyi Cheng7, Abdulaziz Shamsah1,8, Robert Griffin6, C. Karen Liu9, Abderrahmane Kheddar10,11, Xue Bin Peng12, YukeZhu13,14, Guanya Shi15, Quan Nguyen2, Gordon Cheng3, Huijun Gao*16, and Ye Zhao*1
- 【7】基于风速预测和随机规划的含风电场电力系统动态经济调度孙元章，吴俊，李国杰，何剑(电力系统及发电设备控制和仿真国家重点实验室(清华大学电机系)，北京市 海淀区 100084)
Humanoid Robot Motion Planning Approaches: a Survey
- 【8】Carolina Rutili de Lima1 · Said G. Khan2 · Muhammad Tufail3 · Syed H. Shah4,5 · Marcos R.O.A. Maximo

附录

核心算法伪代码与关键公式

CUMCM 2026 Problem B - 众擎 PM01 人形机器人格斗分析

附录 A: 问题一 - 拉格朗日动力学与攻击力指标

A.1 正运动学算法

输入: 关节角度 $\mathbf{q} \in \mathbb{R}^{23}$, 机器人模型 Robot

输出: 各连杆世界变换 $\mathbf{T}_{\text{world}} \in \mathbb{R}^{28 \times 4 \times 4}$, 质心位置 $\mathbf{p}_{\text{com}} \in \mathbb{R}^{28 \times 3}$

算法 1: 正运动学递推

```
T_world[0] ← I_{4×4} // 基座固定
for i = 1 to 27:
    parent ← links[i].parent
    j ← links[i].joint_idx
    if j ≥ 0:
        R_joint ← AxisAngleToRotation(links[i].joint_axis, q[j])
    else:
        R_joint ← I_{3×3}
    T_joint ← [R_joint, 0; 0^T, 1]
    T_world[i] ← T_world[parent] · T_fixed[i] · T_joint
    p_com[i] ← (T_world[i] · [com_offset; 1])_{1:3}
return T_world, p_com
```

A.2 质量矩阵 $\mathbf{M}(\mathbf{q})$ 计算

$$\text{公 式 : } \mathbf{M}_{\{ij\}} = \sum_{k=1}^{28} [\mathbf{m}_k \cdot \mathbf{J}_{\{v,k\}}^T \cdot \mathbf{J}_{\{v,k\}} + \mathbf{J}_{\{\omega,k\}}^T \cdot \mathbf{R}_k \cdot \mathbf{I}_k \cdot \mathbf{R}_k^T \cdot \mathbf{J}_{\{\omega,k\}}] + \mathbf{J}_{\{m,j\}} \cdot \delta_{\{ij\}}$$

其中:

$\mathbf{m}_k$: 第 k 个连杆质量

$\mathbf{J}_{\{v,k\}}$: 第 k 个连杆线速度雅可比 (3×23)

$\mathbf{J}_{\{\omega,k\}}$: 第 k 个连杆角速度雅可比 (3×23)

$\mathbf{R}_k$: 第 k 个连杆世界坐标系旋转矩阵

$\mathbf{I}_k$: 第 k 个连杆体坐标系转动惯量对角阵

$\mathbf{J}_{\{m,j\}}$: 第 j 个关节电机等效惯量

算法 2: 质量矩阵组装

```

M ← zeros(23, 23)
for k = 0 to 27:
    R_k ← T_world[k][:3, :3]
    I_world_k ← R_k · diag(inertia_diag[k]) · R_k^T
    M ← M + m_k · (J_v[k]^T · J_v[k])
    M ← M + J_omega[k]^T · I_world_k · J_omega[k]
for j = 0 to 22:
    M[j,j] ← M[j,j] + motor_inertia[j]
M ← (M + M^T) / 2 // 强制对称
return M

```

A.3 重力矩向量 $g(q)$ 计算

公式: $g_i = -\sum_{k=1}^{28} m_k \cdot g^T \cdot J_{v,k}[:,i]$

其中 $g = [0, 0, -9.81]^T$ 为重力加速度向量。

算法 3: 重力矩计算

```

g_world ← [0, 0, -9.81]^T
g ← zeros(23)
for i = 0 to 22:
    for k = 0 to 27:
        g[i] ← g[i] - m_k · g_world^T · J_v[k][:, i]
return g

```

A.4 $C(q, \dot{q}) \cdot \dot{q}$

使用克里斯托弗尔符号:

$$c_{\{ijk\}} = 0.5 \cdot (\partial M_{\{ij\}} / \partial q_k + \partial M_{\{ik\}} / \partial q_j - \partial M_{\{jk\}} / \partial q_i)$$

$$C \cdot \dot{q}_i = \sum_j \sum_k c_{\{ijk\}} \cdot \dot{q}_j \cdot \dot{q}_k$$

算法 4: 数值微分法计算科里奥利力

```

for k = 0 to 22:
    q_plus ← q; q_plus[k] += δ
    q_minus ← q; q_minus[k] -= δ
    M_plus ← ComputeMassMatrix(q_plus)
    M_minus ← ComputeMassMatrix(q_minus)
    dM_dq[k] ← (M_plus - M_minus) / (2δ)

```

```

C_qdot ← zeros(23)
for i = 0 to 22:
    for j = 0 to 22:
        c_ij ← 0
        for k = 0 to 22:
            c_ijk ← 0.5 · (dM_dq[k][i,j] + dM_dq[j][i,k] - dM_dq[i][j,k])
        c_ij ← c_ij + c_ijk · q̇[k]
    C_qdot[i] ← C_qdot[i] + c_ij · q̇[j]
return C_qdot

```

A.5 攻击力指标计算

输入: $(q, \dot{q})$, M , J_v
 输出: 末端速度 v_{ee} , 等效质量 m_e , 有效动能 E_k , 有效动量 p_m

算法 5: 攻击力指标

```

v_ee ← J_v · q̇ //
v_speed ← ||v_ee|| // 速度大小
u ← v_ee / v_speed // 打击方向单位向量

M_inv ← M^{-1}
Λ^{-1} ← J_v · M_inv · J_v^T // 操作空间惯量逆 (3×3)
m_e ← 1 / (u^T · Λ^{-1} · u) // 等效质量

E_k ← 0.5 · m_e · v_speed^2 // 有效动能
p_m ← m_e · v_speed // 有效动量
return v_ee, v_speed, m_e, E_k, p_m

```

附录 B: 问题二 - 攻防博弈与纳什均衡

B.1 攻防效用矩阵构建

公式: $U_{ij} = H_{ij} - C_i - R_{ij}$

其中:

H_{ij} : 有效伤害 = $E_k^{\text{peak}} \cdot (1 - \text{defense_reduction}_{ij})$

C_i : 能量代价 = $\int ||\tau|| dt$ (归一化)

R_{ij} : 反击风险 (反击类防御时非零)

算法 6: 效用矩阵计算

```
// 1. 能量代价
for i = 0 to 12:
     $\tau\_norm \leftarrow \|\tau\_i(t)\|_2$  (各时间步)
     $C[i] \leftarrow \text{TrapezoidIntegral}(\tau\_norm, dt)$ 
 $C\_norm \leftarrow \text{Normalize}(C, [0,1])$ 

// 2. 有效伤害矩阵
 $E\_k\_base \leftarrow [\text{peak\_E\_k for each attack}]$ 
for i = 0 to 12:
    for j = 0 to 21:
        reduction  $\leftarrow \text{BaseReduction}[\text{defense\_type}[j]]$ 
        reduction  $\leftarrow \text{reduction} \cdot \text{DirectionMatchFactor}(i, j)$ 
         $H[i,j] \leftarrow E\_k\_base[i] \cdot (1 - \text{reduction})$ 

// 3. 反击风险
for i = 0 to 12:
    for j = 0 to 21:
        if defense_type[j] == '反击':
             $R[i,j] \leftarrow \text{CounterAttackRisk}(i, j)$ 

 $U \leftarrow H - C\_norm[:, \text{None}] - R$ 
return U, H, C_norm, R
"""
```

"""

B.2 纯策略纳什均衡 (鞍点) 搜索

鞍点条件: $U[i^*, j] \leq U[i^*, j^*] \leq U[i, j^*]$ 对所有 i, j 成立

算法 7: 鞍点搜索

```
row_mins  $\leftarrow \min_j(U[i,j])$  for each i // 进攻方最坏情况
col_maxs  $\leftarrow \max_i(U[i,j])$  for each j // 防守方最坏情况

maximin  $\leftarrow \max_i(\text{row\_mins})$  // 极大极小值
minimax  $\leftarrow \min_j(\text{col\_maxs})$  // 极小极大值

saddle_points  $\leftarrow []$ 
if  $|\text{maximin} - \text{minimax}| < \epsilon$ :
    for each (i,j):
        if  $|U[i,j] - \text{maximin}| < \epsilon$  and  $|U[i,j] - \text{row\_mins}[i]| < \epsilon$ 
           and  $|U[i,j] - \text{col\_maxs}[j]| < \epsilon$ :
            saddle_points.append((i,j))
```

```

        return saddle_points, (maximin == minimax)
    """

```

```

    """

```

B.3 混合策略纳什均衡 (线性规划)

当纯策略均衡不存在时, 求解混合策略:

设 $x \in R^{\{13\}}$ 为进攻方策略概率, $y \in R^{\{22\}}$ 为防守方策略概率

进攻方问题:

```

max    v
s.t.  $U^T \cdot x \geq v \cdot 1_{\{22\}}$ 
       $1^T \cdot x = 1, x \geq 0$ 

```

防守方问题:

```

min    w
s.t.  $U \cdot y \leq w \cdot 1_{\{13\}}$ 
       $1^T \cdot y = 1, y \geq 0$ 

```

算法 8: 线性规划求解混合策略

```

// 进攻方
c ← [-1, 0, ..., 0]                // 目标函数系数
A_ub ← [-1_{22}, -U^T]             // 不等式约束
b_ub ← zeros(22)
A_eq ← [0, 1_{13}^T]               // 等式约束
b_eq ← [1]
bounds ← [(0, None)] + [(0, 1)] * 13
x_opt, v ← linprog(c, A_ub, b_ub, A_eq, b_eq, bounds)

return x_opt[1:], v                // 最优混合策略, 博弈值
"""

```

附录 C: 问题三 - MDP 最优格斗策略

C.1 状态空间与动作空间

状态: $s = (\text{score}, \text{stamina}, \text{distance}, \text{time_phase}, \text{opponent}) \in S$

score $\in \{0, 1, 2\}$: 净胜分等级 (劣势/均势/优势)

stamina $\in \{0, 1, 2\}$: 体力等级 (低/中/高)

distance $\in \{0, 1, 2\}$: 距离等级 (近/中/远)

time_phase $\in \{0, 1, 2\}$: 时间阶段 (前/中/后)

opponent $\in \{0,1\}$: 对手姿态 (进攻/防守)

$$|S| = 3 \times 3 \times 3 \times 3 \times 2 = 162$$

动作: $a \in A = A_attack \cup A_defense$

A_attack: 13 种攻击动作 (索引 0-12)

A_defense: 22 种防御动作 (索引 13-34)

$$|A| = 35$$

.....

.....

C.2 奖励函数 $R(s, a, s')$

公 式 : $R(s,a,s') = 10 \cdot \Delta score \cdot \tau_t + 5 \cdot \Delta situation \cdot \tau_t + 3 \cdot d_ctrl - 20 \cdot cost_a + aggr(s) \cdot \Delta score$

其中:

$\Delta score$: 分差等级变化 (s' 的 score - s 的 score)

τ_t : 时间紧迫度权重 [0.5, 1.0, 2.0]

$\Delta situation$: 对手姿态变化 (+1 对手转防守, -1 对手转进攻)

d_ctrl : 距离控制奖励 (攻击时距离匹配+1, 不匹配-0.5)

$cost_a$: 动作体力消耗

$aggr(s)$: 分差策略系数 [1.5, 1.0, 0.8] (落后时更激进)

算法 9: 奖励计算

```
R ← 10 × (score_next - score_now) ×  $\tau$ [time_phase]
    + 5 × situation_change ×  $\tau$ [time_phase]
    + 3 × distance_control
    - 20 × stamina_cost[action]
    + aggression[score_now] × (score_next - score_now)
```

```
return R
```

.....

.....

C.3 专家规则转移概率 $P(s'|s,a)$

对每个 (s,a) , 基于专家规则定义 2-4 个可能的下一状态及概率。

算法 10: 转移概率构建

```
P ← zeros(162, 35, 162)
for s = 0 to 161:
    (sc, st, di, ti, op) ← DecodeState(s)
    for a = 0 to 34:
        if is_attack(a):
            power ← ATTACK_POWER[a]
```

```

        dist_match ← DistanceMatchFactor(di, a)
        p_hit ← BaseHitProb × power × dist_match × (1 + 0.1×st)

        // 命中: 分差提升
        s_hit ← EncodeState(min(sc+1,2), st_next, di_next, ti_next,
op_next)

        P[s,a,s_hit] += p_hit

        // 未命中: 体力消耗
        s_miss ← EncodeState(sc, st_next, di, ti_next, op)
        P[s,a,s_miss] += (1 - p_hit)
    else:
        defense ← DEFENSE_STRENGTH[a-13]
        p_block ← BaseBlockProb × defense × (1 + 0.05×st)

        // 成功防御: 对手转防守
        s_block ← EncodeState(sc, st_next, di_next, ti_next, 1)
        P[s,a,s_block] += p_block

        // 防御失败: 保持原态
        s_fail ← EncodeState(sc, st_next, di, ti_next, op)
        P[s,a,s_fail] += (1 - p_block)

    return P
"""

```

C.4 值迭代求解最优策略

 Bellman 最优方程: $V^*(s) = \max_a [R(s,a) + \gamma \cdot \sum_{s'} P(s'|s,a) \cdot V^*(s')]$

算法 11: 向量化值迭代

```

V ← zeros(|S|)
for iter = 1 to MAX_ITER:
    // 向量化 Q 值计算
    PV ← einsum('ijk,k->ij', P, V)           // P @ V, 形状 (162, 35)
    Q ← R + γ × PV                             // Q 值矩阵
    V_new ← max_a(Q, axis=1)                   // 最优价值

    δ ← ||V_new - V||_∞
    V ← V_new
    if δ < tol: break

// 提取最优策略
PV ← einsum('ijk,k->ij', P, V)

```

```

     $Q \leftarrow R + \gamma \times PV$ 
     $\pi^*(s) \leftarrow \operatorname{argmax}_a(Q[s,a])$  // 最优策略
    return  $V, \pi^*$ 

```

C.5 蒙特卡洛策略评估

算法 12: MC 仿真评估

```

returns  $\leftarrow \text{zeros}(|S|)$ 
counts  $\leftarrow \text{zeros}(|S|)$ 
for  $s_0 = 0$  to 161:
    for  $ep = 1$  to  $N$ :
         $s \leftarrow s_0$ , total_reward  $\leftarrow 0$ , discount  $\leftarrow 1$ 
        for  $t = 1$  to ROUND_TIME:
             $a \leftarrow \pi^*(s)$ 
             $s' \leftarrow \text{SampleFromDistribution}(P[s,a,:])$ 
             $r \leftarrow R[s,a]$ 
            total_reward  $+=$  discount  $\times r$ 
            discount  $\times= \gamma$ 
             $s \leftarrow s'$ 
        returns[ $s_0$ ]  $+=$  total_reward
        counts[ $s_0$ ]  $+= 1$ 
V_mc  $\leftarrow$  returns / counts
return V_mc

```

附录 D: 问题四 - BO3 资源调度优化

D.1 两层层次化模型架构

第一层: 局内 MDP (单局比赛内的格斗与资源决策)

状态: $s_{\text{game}} = (h, f, t, sd) \in S_{\text{game}}$

$h \in \{0, \dots, 4\}$: 健康度等级 (1.0, 0.8, 0.6, 0.4, 0.2)

$f \in \{0, \dots, 3\}$: 故障等级 (无/轻/中/重)

$t \in \{0, \dots, 9\}$: 剩余时间 (每档 30s)

$sd \in \{0, \dots, 4\}$: 比分差 (-2, -1, 0, +1, +2)

$|S_{\text{game}}| = 5 \times 4 \times 10 \times 5 = 1000$

动作: $a \in A_{\text{game}} = A_{\text{combat}} \cup A_{\text{resource}}$

A_{combat} : 35 种格斗动作 (索引 0-34)

A_{resource} : 3 种资源动作 (人工复位/战术暂停/紧急维修, 索引 35-37)

$$|A_game| = 38$$

第二层: BO3 马尔可夫链 (跨局资源分配)

状态: $s_{bo3} = (w_m, w_o, r_res, r_tact, r_repair, h_start)$

$w_m \in \{0,1\}$: 我方胜局数

$w_o \in \{0,1\}$: 对方胜局数

$r_res \in \{0,1,2\}$: 剩余人工复位次数

$r_tact \in \{0,1,2\}$: 剩余战术暂停次数

$r_repair \in \{0,1\}$: 剩余紧急维修次数

$h_start \in \{0,...,4\}$: 本局起始健康度

$$|S_{bo3}| = 4 \times 3 \times 3 \times 2 \times 5 = 360$$

.....

.....

D.2 故障发生模型 (泊松过程)

$$\text{公式: } \lambda(t,h) = \lambda_o \cdot e^{\alpha \cdot (1-h_val)} \cdot (1 + 0.3 \cdot t/N_TIME) \cdot (1 + 0.8 \cdot f)$$

其中:

$\lambda_o = 0.015$: 基础故障率

$\alpha = 1.5$: 健康度敏感系数

h_val : 健康度实际值 (0.2~1.0)

t : 当前时间档

f : 当前故障等级

算法 13: 故障等级转移

$p_fault \leftarrow \min(\lambda_o \times \exp(\alpha \times (1-h_val)) \times time_factor \times fault_factor, 0.5)$

if random() < p_fault:

 // 故障发生, 等级提升

 if $f == 0$: $f_next \leftarrow \text{Bernoulli}(0.7) ? 1 : 2$

 elif $f == 1$: $f_next \leftarrow \text{Bernoulli}(0.6) ? 2 : 3$

 elif $f == 2$: $f_next \leftarrow \text{Bernoulli}(0.8) ? 3 : 2$

 else: $f_next \leftarrow 3$

else:

$f_next \leftarrow f$

return f_next

.....

.....

D.3 局内值迭代

与问题三相同的值迭代框架, 但状态空间扩大到 1000。

算法 14: 局内值迭代

```

V_game ← zeros(1000)
for iter = 1 to 3000:
    PV ← einsum('ijk,k->ij', P_game, V_game)    // (1000, 38)
    Q ← R_game + 0.98 × PV
    V_new ← max_a(Q, axis=1)
    δ ← ||V_new - V_game||_∞
    V_game ← V_new
    if δ < 1e-4: break
    π_game*(s) ← argmax_a(Q[s,a])
return V_game, π_game*

```

.....

D.4 局胜率计算 (前向概率传播)

从初始状态 (h, f=0, t=0, sd=0) 出发, 按最优策略传播概率。

算法 15: 局胜率计算

```

function ComputeGameWinProb(h_start):
    s0 ← EncodeGameState(h_start, 0, 0, 2)    // sd=2 表示平局开始
    dist ← zeros(1000)
    dist[s0] ← 1.0

    for step = 1 to 10:
        new_dist ← zeros(1000)
        for s = 0 to 999:
            if dist[s] < 1e-15: continue
            a ← π_game*(s)
            new_dist += dist[s] × P_game[s, a, :]
        dist ← new_dist

    // 统计终端状态
    p_win ← 0
    for s = 0 to 999:
        (h, f, t, sd) ← DecodeGameState(s)
        if t ≥ 9 and sd ≥ 3:
            p_win += dist[s]
    return p_win

```

.....

D.5 BO3 递归胜率计算

公式: $P_win(w_m, w_o, R, h) = pw \cdot P_win(w_m+1, w_o, R', h') + pl \cdot P_win(w_m,$

$w_o+1, R', h')$

其中 pw, pl 为当前健康度下的局胜率, R' 为资源消耗后的状态, h' 为下局健康度。

算法 16: 递归 BO3 胜率 (带记忆化)

```
@lru_cache(maxsize=None)
function CalcBo3Prob(w_m, w_o, r_res, r_tact, r_repair, h_start):
    if w_m ≥ 2: return 1.0                // 我方胜 BO3
    if w_o ≥ 2: return 0.0                // 对方胜 BO3

    pw ← GameWinProb[h_start].p_win
    pl ← GameWinProb[h_start].p_lose
    h_next ← min(h_start + 1, 4)          // 局间健康退化

    // 基线: 不使用资源
    p_win ← pw × CalcBo3Prob(w_m+1, w_o, R..., h_next)
        + pl × CalcBo3Prob(w_m, w_o+1, R..., h_next)

    // 尝试使用每种可用资源
    for each available resource:
        R' ← R with resource consumed
        if resource == 复位:
            pw_r ← min(pw + boost[scenario], 0.95)    // 场景相关 boost
        elif resource == 暂停:
            pw_r ← GameWinProb[max(h-1, 0)].p_win    // 健康恢复
        elif resource == 维修:
            pw_r ← pw × penalty[scenario]             // 牺牲本局
            h_repair ← 1                               // 下局满血

        p_r ← ComputeBo3ProbWithResource(...)
        if p_r > p_win + ε:
            p_win ← p_r
            best_action ← resource

    return p_win
"""
"""
```

D.6 最优资源使用策略搜索

算法 17: 枚举搜索最优策略

```
strategy ← {}
for each BO3 state (w_m, w_o, R, h):
    if w_m ≥ 2 or w_o ≥ 2: continue
```

```

// 计算基线胜率
baseline  $\leftarrow$  CalcBo3Prob(w_m, w_o, R, h)
best_prob  $\leftarrow$  baseline
best_action  $\leftarrow$  '无'

// 枚举所有可用资源
for each available resource in R:
    R'  $\leftarrow$  R with resource consumed
    prob  $\leftarrow$  CalcBo3ProbWithResource(w_m, w_o, R', h, resource)
    if prob > best_prob +  $\epsilon$ :
        best_prob  $\leftarrow$  prob
        best_action  $\leftarrow$  resource

strategy[state]  $\leftarrow$  {action: best_action, win_prob: best_prob}
return strategy

```

"""

D.7 蒙特卡洛 BO3 仿真验证

算法 18: MC BO3 仿真

```

results  $\leftarrow$  {win_2_0: 0, win_2_1: 0, lose_0_2: 0, lose_1_2: 0}

for ep = 1 to 20000:
    w_m  $\leftarrow$  0, w_o  $\leftarrow$  0
    R  $\leftarrow$  (2, 2, 1) // 满资源
    h  $\leftarrow$  0 // 满血

    for game = 1 to 3:
        if w_m  $\geq$  2 or w_o  $\geq$  2: break

        // 查找当前状态的最优策略
        action  $\leftarrow$  strategy[(w_m, w_o, R, h)]

        // 根据资源类型调整本局胜率
        pw  $\leftarrow$  GameWinProb[h].p_win
        if action == '维修':
            pw  $\leftarrow$  pw  $\times$  0.15 // 牺牲本局
        elif action == '暂停':
            pw  $\leftarrow$  GameWinProb[max(h-1,0)].p_win // 健康恢复
        elif action == '复位':
            pw  $\leftarrow$  min(pw + boost, 0.95) // 清除故障

```


序号	动作名称	E_k(J)	m_e(kg)	v(m/s)	p(N·s)
1	左直拳	12.3	2.1	3.4	7.2
2	右直拳	13.8	2.3	3.5	8.0
3	左摆拳	15.2	2.0	3.9	7.8
4	右摆拳	16.1	2.1	3.9	8.2
5	左上勾拳	14.5	2.2	3.6	8.0
6	右上勾拳	15.0	2.3	3.6	8.3
7	左掌击	10.8	1.9	3.4	6.5
8	右肘击	18.5	2.5	3.8	9.5
9	左膝击	22.3	3.2	3.7	11.8
10	右前蹬	25.1	3.5	3.8	13.3
11	右侧踢	28.7	3.8	3.9	14.8
12	右回旋踢	35.2	4.1	4.1	16.8
13	右后踢	30.5	3.9	3.9	15.2

(注: 数值为典型值, 实际结果见 results.npz)

表 E.3: 问题四资源效果模型

资源	领先(1-0)	平局(0-0)	落后(0-1)
人工复位	+0.02 胜率提升	+0.05 胜率提升	+0.08 胜率提升
战术暂停	恢复 1 档健康	恢复 1 档健康	恢复 1 档健康
紧急维修	本局×0.12	本局×0.12	本局×0.25

```
if __name__ == '__main__':
    print("附录算法文件 - 可直接嵌入论文附录")
    print("包含: 问题一(拉格朗日动力学), 问题二(博弈论), 问题三(MDP), 问题四(BO3 调度)")
```